



Sanjivani Rural Education Society's
Sanjivani College of Engineering, Kopargaon-423 603
(An Autonomous Institute, Affiliated to Savitribai Phule Pune University, Pune)
NAAC 'A' Grade Accredited, ISO 9001:2015 Certified

Department of Computer Engineering

(NBA Accredited)

Subject- Object Oriented Programming (CO212)
Unit 1 – Fundamentals of OOP
Topic – 1.7 Arrays in C++

Prof. V.N.Nirgude
Assistant Professor

E-mail :

nirgudevikascomp@sanjivani.org.in

Contact No: 9975240215



WHAT IS AN ARRAY?

- An array is a group of consecutive memory locations with same name and data type.
- Simple variable is a single memory location with unique name and a type.
- But an Array is a collection of different adjacent memory locations.
- All these memory locations have one collective name and data type.
- The total number of elements in the array is called length of an array.
- The element of an array is accessed with reference to its position in array, that is called as index or subscript.



DECLARATION OF AN ARRAY

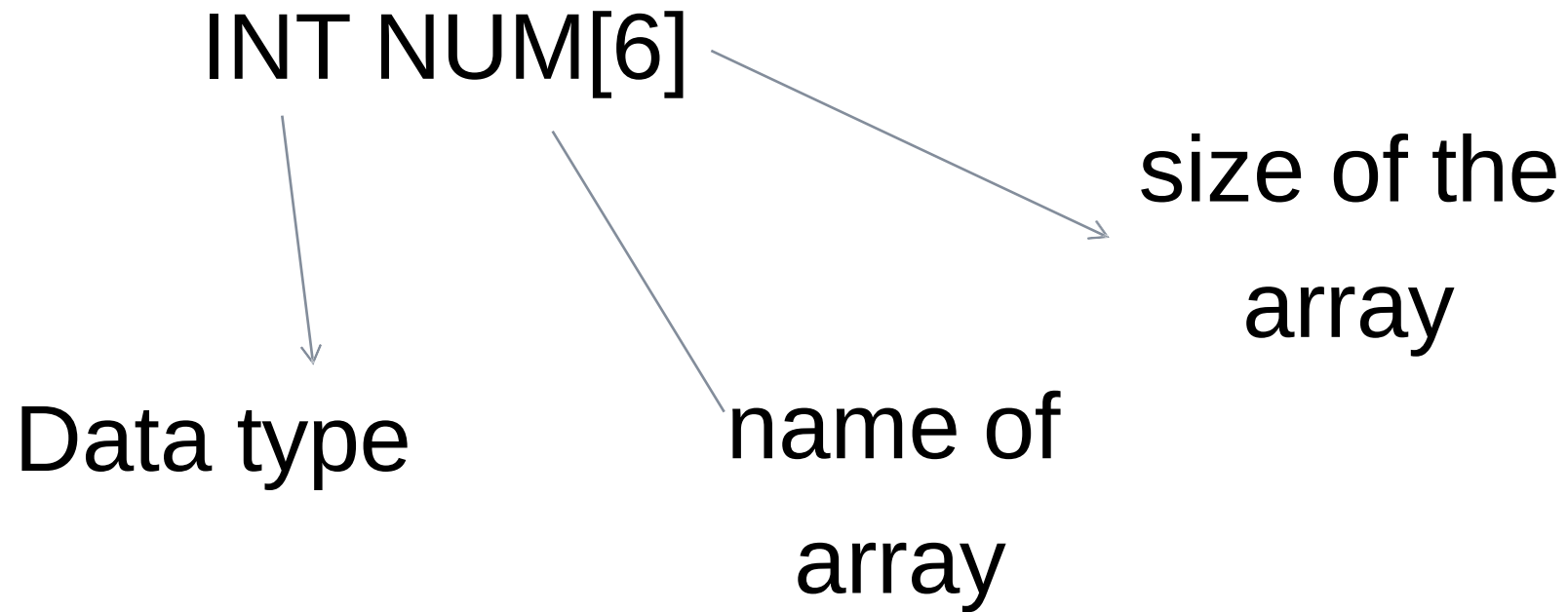
- Like a regular variable, an array must be declared before it is used.

Data_Type Array_Name[size];

- **Data_Type** is a valid type (like `int`, `float`...)
- **Array_Name** is a valid identifier
- **size** field (which is always enclosed in square brackets `[]`), specifies how many number of elements the array has to contain.



EXAMPLE OF ARRAY DECLARATION





ADVANTAGES OF ARRAY

- Arrays can store a large number of value with single name.
- Arrays are used to process many value easily and quickly.
- The values stored in an array can be sorted easily.
- The search process can be applied on arrays easily.



SAMPLE PROGRAM

```
#include <iostream>
using namespace std;

void main ()
{
    int array[5];
    array[0] = 10;
    array[1] = 20;
    array[2] = 30;
    array[3] = 40;
    array[4] = 20;

    cout<<"Array contents:" <<endl;
    for ( int n=0 ; n<5 ; n++ )
    {   cout <<"array[" <<n <<" ] = " <<array[n] <<endl; }
}
```

```
Array contents:
array[0] = 10
array[1] = 20
array[2] = 30
array[3] = 40
array[4] = 20
```

TYPES OF ARRAY:

- One-dimensional array
- Two-dimensional array
- Multi-dimensional array



ONE-DIMENSIONAL ARRAY

- A one dimensional array is one in which one subscript/size specification is needed to specify size of array.

Declaration :

Data_type array_name [size];

Eg:

```
int num[10];
```




MEMORY REPRESENTATION

num[0] num[1] num[2] num[3] num[4] num[5] num[6] num[7] num[8] num[9]

39	56	23	98	6	56	9	2	54	67
----	----	----	----	---	----	---	---	----	----

2000 2002 2004 2006 2008 2010 2012 2014 2016 2018



starting Address of location

Total memory in bytes that an array is occupied:

Total memory of array=size of array*size of(base address)

Hear,

$$=10*2$$

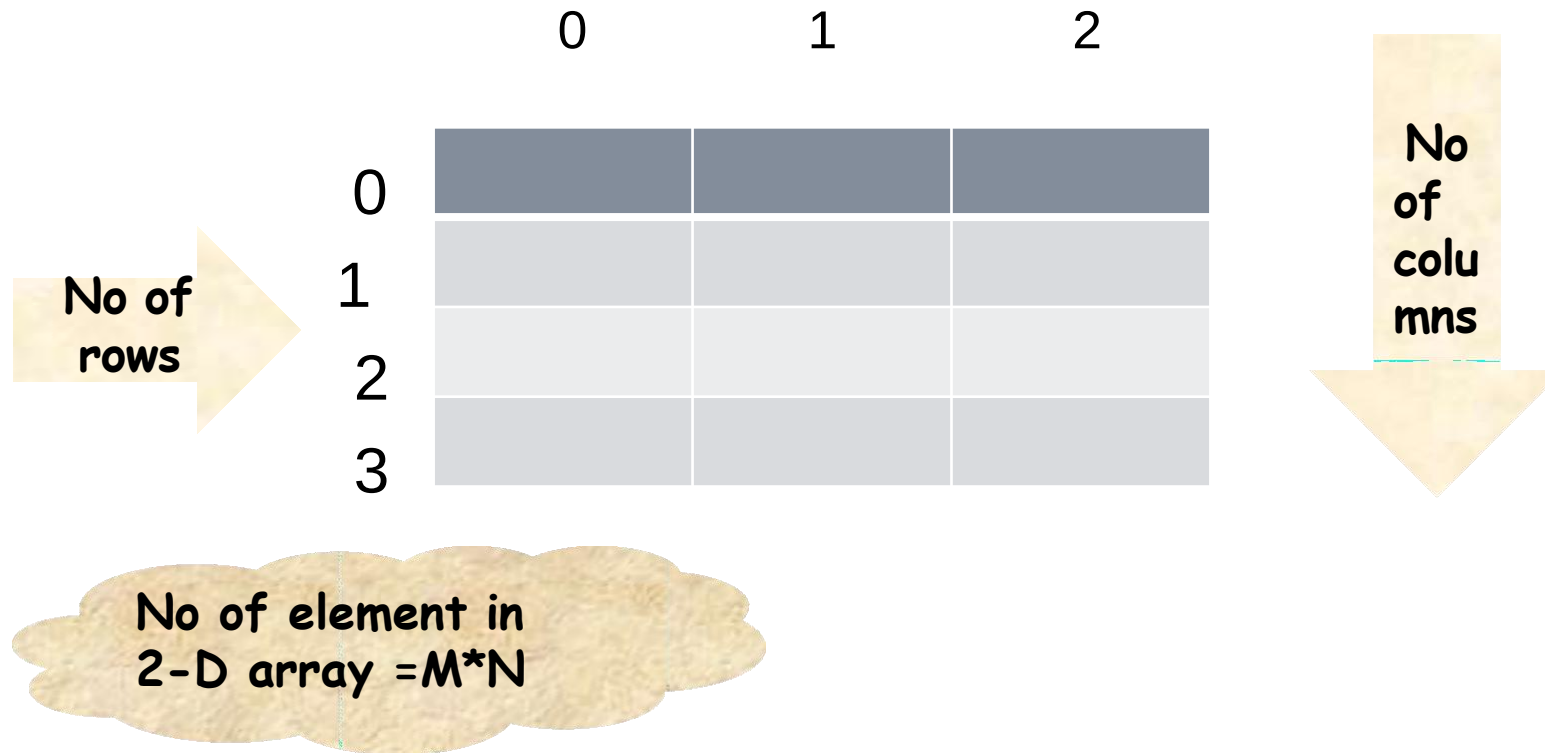
$$=20 \text{ bytes}$$



TWO-DIMENSIONAL ARRAY

A 2-d array is an array in which each element is specified with two indices.

i.e `int num[4][3]`





EXAMPLE PROGRAM

```
#include<iostream>
using namespace std;

void main()
{
    int array[4][3]={ {1,2,3}, {4,5,6}, {7,8,9}, {10, 11, 12} };
    for(int i=0; i<4;i++)
        for(int n=0; n<3;n++)
            { cout<<"array [" <<i <<" ] [" <<n <<" ] = " <<array[i][n] <<endl; }
}
```

```
array [0] [0] = 1
array [0] [1] = 2
array [0] [2] = 3
array [1] [0] = 4
array [1] [1] = 5
array [1] [2] = 6
array [2] [0] = 7
array [2] [1] = 8
array [2] [2] = 9
array [3] [0] = 10
array [3] [1] = 11
array [3] [2] = 12
```

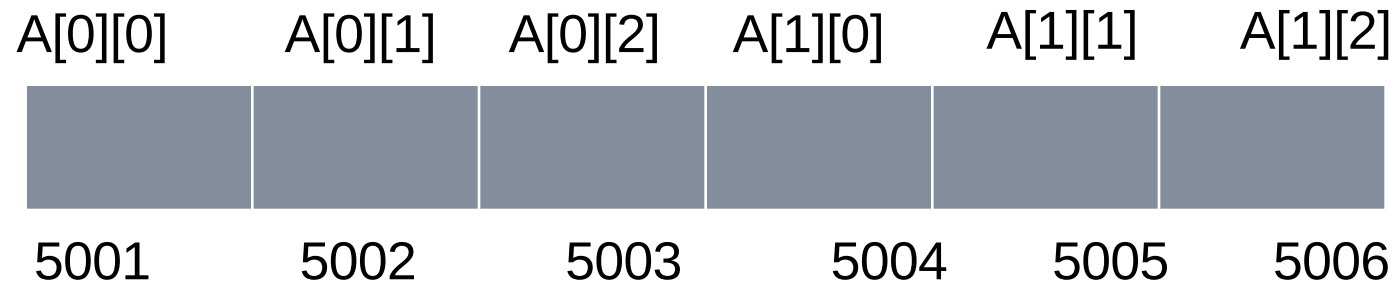


MEMORY REPRESENTATION

Total bytes= no of rows*no of columns*size of(base address)

Memory representation in 2-D array:

char A [2][3]





MULTI-DIMENSIONAL ARRAY

- It is an array with dimensions more than two.

- Declaration:

Data_type array_name [a][b][c][d][e][f].....[n];

- Array with 3 or more dimensional type are not often use because of huge memory requirement and complexity involved.

Thank You..